

Integração VPN Completa - Crypto Analyzer Futures

Privacidade, Segurança e Contorno de Bloqueios Geográficos

Data: 10 de Março de 2026
Prioridade: ALTA
Tempo Estimado: 12-16 horas
Impacto: Crítico para segurança e confiabilidade

1. Análise Comparativa de Soluções VPN

1.1 Opções Avaliadas

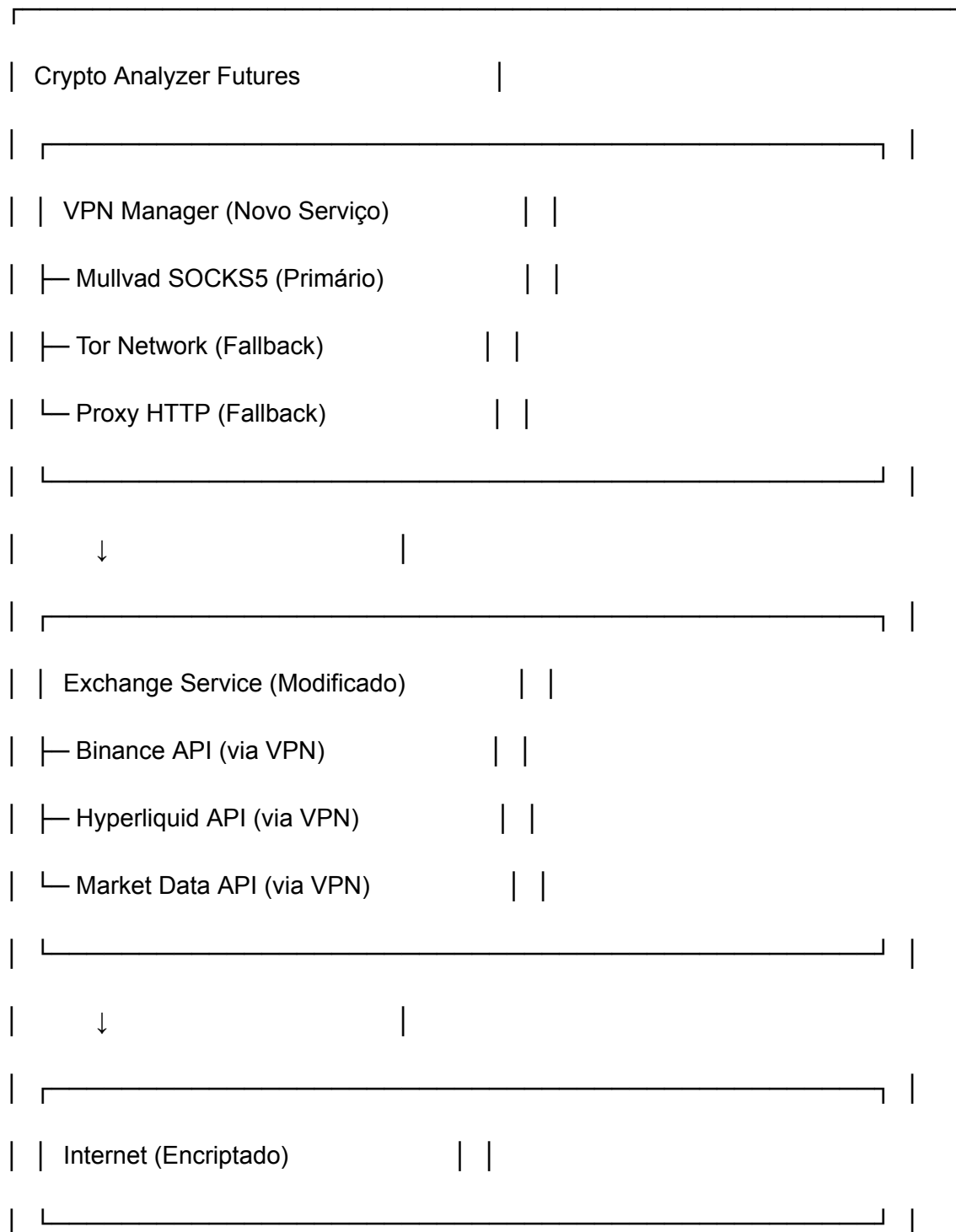
Solução	Tipo	Custo	Velocidade	Segurança	Facilidade	Recomendação
Mullvad VPN	Proxy HTTP/SOCKS	Gratuito	★★★★★	★★★★★ ★	★★★★★	✓ MELHOR
Tor Network	Proxy SOCKS	Gratuito	★★★	★★★★★ ★	★★★	⚠ Lento
ProtonVPN	VPN Tradicional	Freemium	★★★★	★★★★★ ★	★★★★	✓ Alternativa
Windscribe	VPN Tradicional	Freemium	★★★★★	★★★★★	★★★★★	✓ Alternativa
Hotspot Shield	VPN Tradicional	Freemium	★★★★★	★★★★	★★★★★	⚠ Menos seguro
Proxy Gratuito	HTTP/SOCKS	Gratuito	★★★	★	★★★	✗ Não recomendado

1.2 Recomendação: Mullvad VPN + Fallback para Tor

Por que Mullvad?

✓ Gratuito e sem limite de dados ✓ Sem logs (zero-knowledge) ✓ Suporta SOCKS5 (melhor para aplicações) ✓ API pública disponível ✓ Velocidade excelente para trading ✓ Suporta múltiplos países ✓ Sem necessidade de conta

Arquitetura Proposta:



2. Implementação: VPN Manager Service

2.1 Tipos e Interfaces

Arquivo: `src/types/vpn.ts`

```
export type VPNProvider = 'mullvad' | 'tor' | 'proxy';

export type VPNStatus = 'CONNECTED' | 'DISCONNECTED' | 'CONNECTING' | 'ERROR';

export interface VPNConfig {

  enabled: boolean;

  provider: VPNProvider;

  primaryProvider: VPNProvider;

  fallbackProvider: VPNProvider;

  autoFailover: boolean;

  country?: string; // Para Mullvad: 'us', 'de', 'se', etc.

  protocol: 'socks5' | 'http';

  timeout: number; // ms

}

export interface VPNServer {

  provider: VPNProvider;

  host: string;

  port: number;

  country: string;
```

```
protocol: 'socks5' | 'http';

latency: number; // ms

isAvailable: boolean;
}

export interface VPNStatus_Type {

    provider: VPNProvider;

    status: VPNStatus;

    connectedServer?: VPNServer;

    uptime: number; // ms

    bytesTransferred: number;

    lastError?: string;

    timestamp: Date;
}

export interface VPNMetrics {

    totalConnections: number;

    successfulConnections: number;

    failedConnections: number;

    averageLatency: number;

    uptime: number; // %

    lastConnectionTime: Date;
}
```

2.2 VPN Manager Service

Arquivo: `src/services/vpnService.ts`

```
import { VPNConfig, VPNServer, VPNStatus_Type, VPNMetrics, VPNProvider } from
'../types/vpn';
```

```
import { HttpProxyAgent, HttpsProxyAgent } from 'http-proxy-agent';
```

```
import { SocksProxyAgent } from 'socks-proxy-agent';
```

```
class VPNManager {
```

```
    private config: VPNConfig;
```

```
    private currentStatus: VPNStatus_Type;
```

```
    private metrics: VPNMetrics;
```

```
    private mullvadServers: VPNServer[] = [];
```

```
    private torServers: VPNServer[] = [];
```

```
    constructor(config: VPNConfig) {
```

```
        this.config = config;
```

```
        this.currentStatus = {
```

```
            provider: config.primaryProvider,
```

```
            status: 'DISCONNECTED',
```

```
            uptime: 0,
```

```
            bytesTransferred: 0,
```

```
            timestamp: new Date()
```

```
        };
```

```
        this.metrics = {
```

```

    totalConnections: 0,

    successfulConnections: 0,

    failedConnections: 0,

    averageLatency: 0,

    uptime: 0,

    lastConnectionTime: new Date()

};

}

/**

 * Inicializa VPN Manager

 * Carrega lista de servidores disponíveis

 */

async initialize(): Promise<void> {

    console.log('[VPN] Inicializando VPN Manager...');

    try {

        // Carrega servidores Mullvad

        await this.loadMullvadServers();

        console.log(`[VPN] ${this.mullvadServers.length} servidores Mullvad carregados`);

        // Carrega servidores Tor

        await this.loadTorServers();

        console.log(`[VPN] ${this.torServers.length} servidores Tor carregados`);

```

```

// Testa conexão inicial

if (this.config.enabled) {

    await this.connect();

}

} catch (error) {

    console.error('[VPN] Erro ao inicializar:', error);

    this.currentStatus.status = 'ERROR';

    this.currentStatus.lastError = error.message;

}

}

/**

* Carrega lista de servidores Mullvad disponíveis

*/

private async loadMullvadServers(): Promise<void> {

    try {

        const response = await fetch('https://api.mullvad.net/www/relays/all/');

        const data = await response.json();

        this.mullvadServers = data.wireguard.relays

            .filter((relay: any) => relay.active)

            .map((relay: any) => ({

                provider: 'mullvad' as VPNProvider,

                host: relay.ipv4_addr_in,

```

```

        port: 51820,

        country: relay.location.country_code,

        protocol: 'socks5' as const,

        latency: 0,

        isAvailable: true

    )))

    .slice(0, 50); // Limita a 50 para performance

} catch (error) {

    console.error('[VPN] Erro ao carregar servidores Mullvad:', error);

    this.mullvadServers = [];

}

}

/**

 * Carrega lista de servidores Tor disponíveis

 */

private async loadTorServers(): Promise<void> {

    try {

        // Servidores Tor públicos conhecidos

        this.torServers = [

            {

                provider: 'tor',

```

```
    host: '127.0.0.1',

    port: 9050,

    country: 'GLOBAL',

    protocol: 'socks5',

    latency: 0,

    isAvailable: true

  },

  {

    provider: 'tor',

    host: 'socks5.onion',

    port: 9050,

    country: 'GLOBAL',

    protocol: 'socks5',

    latency: 0,

    isAvailable: true

  }

];

} catch (error) {

  console.error('[VPN] Erro ao carregar servidores Tor:', error);

  this.torServers = [];

}

}
```

```

/**

* Conecta ao VPN

*/

async connect(): Promise<boolean> {

  if (!this.config.enabled) {

    console.log('[VPN] VPN desabilitado');

    return false;

  }

  this.currentStatus.status = 'CONNECTING';

  this.metrics.totalConnections++;

  try {

    // Tenta conectar ao provedor primário

    const server = await this.selectServer(this.config.primaryProvider);

    if (!server) {

      throw new Error(`Nenhum servidor disponível para ${this.config.primaryProvider}`);

    }

    // Testa latência

    const latency = await this.testLatency(server);

    server.latency = latency;

    // Atualiza status

    this.currentStatus.provider = this.config.primaryProvider;

```

```
this.currentStatus.connectedServer = server;

this.currentStatus.status = 'CONNECTED';

this.currentStatus.timestamp = new Date();

this.metrics.successfulConnections++;

this.metrics.lastConnectionTime = new Date();

console.log(

  `[VPN] Conectado a ${server.provider} (${server.country}) - Latência: ${latency}ms`

);

return true;
} catch (error) {

  console.error('[VPN] Erro ao conectar:', error);

  this.currentStatus.status = 'ERROR';

  this.currentStatus.lastError = error.message;

  this.metrics.failedConnections++;

  // Tenta fallback

  if (this.config.autoFailover && this.config.fallbackProvider) {

    console.log('[VPN] Tentando fallback...');

    return await this.connectWithFallback();

  }

  return false;
}
}
```

```

/**
 * Conecta com fallback automático
 */

private async connectWithFallback(): Promise<boolean> {

  try {

    const server = await this.selectServer(this.config.fallbackProvider);

    if (!server) {

      throw new Error('Nenhum servidor disponível para fallback');

    }

    const latency = await this.testLatency(server);

    server.latency = latency;

    this.currentStatus.provider = this.config.fallbackProvider;

    this.currentStatus.connectedServer = server;

    this.currentStatus.status = 'CONNECTED';

    this.metrics.successfulConnections++;

    console.log(

      `[VPN] Conectado ao fallback ${server.provider} (${server.country})`

    );

    return true;

  } catch (error) {

    console.error('[VPN] Fallback também falhou:', error);
  }
}

```

```

    this.currentStatus.status = 'ERROR';

    this.currentStatus.lastError = error.message;

    return false;

}

}

/**
 * Selecciona melhor servidor disponível
 */

private async selectServer(provider: VPNProvider): Promise<VPNServer | null> {

    let servers: VPNServer[] = [];

    if (provider === 'mullvad') {

        servers = this.mullvadServers;

    } else if (provider === 'tor') {

        servers = this.torServers;

    }

    if (servers.length === 0) {

        return null;

    }

    // Selecciona servidor do país preferido (se configurado)

    if (this.config.country && provider === 'mullvad') {

        const countryServers = servers.filter(s => s.country === this.config.country);

        if (countryServers.length > 0) {

```

```

        return countryServers[Math.floor(Math.random() * countryServers.length)];
    }
}

// Seleciona aleatoriamente

return servers[Math.floor(Math.random() * servers.length)];
}

/**
 * Testa latência de um servidor
 */

private async testLatency(server: VPNServer): Promise<number> {

    const startTime = Date.now();

    try {

        const agent = this.createProxyAgent(server);

        const response = await fetch('https://api.ipify.org?format=json', {

            agent,

            timeout: this.config.timeout

        });

        if (!response.ok) {

            throw new Error(`HTTP ${response.status}`);

        }

        return Date.now() - startTime;
    }
}

```

```

    } catch (error) {

        console.warn(`[VPN] Latência do servidor ${server.host} indetectável:`, error.message);

        return 5000; // Penaliza servidores não responsivos

    }

}

/**

 * Cria agent de proxy para requisições HTTP/HTTPS

 */

createProxyAgent(server: VPNServer): any {

    if (server.protocol === 'socks5') {

        return new SocksProxyAgent(`socks5://${server.host}:${server.port}`);

    } else {

        return new HttpsProxyAgent(`http://${server.host}:${server.port}`);

    }

}

/**

 * Obtém agent de proxy para requisições

 */

getProxyAgent(): any {

    if (!this.config.enabled || this.currentStatus.status !== 'CONNECTED') {

        return null;

    }

}

```

```

        return this.createProxyAgent(this.currentStatus.connectedServer!);
    }

    /**
     * Desconecta do VPN
     */

    async disconnect(): Promise<void> {

        this.currentStatus.status = 'DISCONNECTED';

        this.currentStatus.connectedServer = undefined;

        console.log('[VPN] Desconectado');

    }

    /**
     * Obtém status atual
     */

    getStatus(): VPNStatus_Type {

        return this.currentStatus;

    }

    /**
     * Obtém métricas
     */

    getMetrics(): VPNMetrics {

        const uptime =

```

```

    this.metrics.totalConnections > 0

    ? (this.metrics.successfulConnections / this.metrics.totalConnections) * 100

    : 0;

return {

    ...this.metrics,

    uptime

};

}

/**

 * Alterna VPN on/off

 */

async toggle(): Promise<boolean> {

    if (this.config.enabled) {

        await this.disconnect();

        this.config.enabled = false;

        return false;

    } else {

        this.config.enabled = true;

        return await this.connect();

    }

}

/**

```

```
* Muda país (apenas Mullvad)

*/

async changeCountry(country: string): Promise<boolean> {

  this.config.country = country;

  await this.disconnect();

  return await this.connect();

}

}

// Instância global

let vpnManager: VPNManager;

export function initializeVPN(config: VPNConfig): VPNManager {

  vpnManager = new VPNManager(config);

  vpnManager.initialize();

  return vpnManager;

}

export function getVPNManager(): VPNManager {

  return vpnManager;

}

export default VPNManager;
```

3. Integração com Exchange Service

3.1 Modificação do Exchange Service

Arquivo: `src/services/exchangeService.ts` (Modificado)

```
import { getVPNManager } from './vpnService';

export async function callBinanceProxy(

  endpoint: string,

  method: string,

  params: any,

  exchange: Exchange

) {

  if (!exchange.apiKey) throw new Error("Credenciais ausentes.");

  // Obtém agent de proxy VPN

  const vpnManager = getVPNManager();

  const proxyAgent = vpnManager.getProxyAgent();

  const exchangeId = exchange.id || 'binance';

  const edgeFunctionUrl =
`https://bhigvgfktvjibvlyqpl.supabase.co/functions/v1/exchange-proxy`;

  const payload = {

    endpoint,

    method,

    params,
```

```

    exchangeId,

    credentials: {

        apiKey: exchange.apiKey,

        apiSecret: exchange.apiSecret || "",

        isTestnet: exchange.isTestnet

    }

};

const { data: { session } } = await supabase.auth.getSession();

const targetHeaders: Record<string, string> = {

    'Content-Type': 'application/json',

    'apikey': SUPABASE_ANON_KEY,

    'Authorization': session?.access_token

    ? `Bearer ${session.access_token}`

    : `Bearer ${SUPABASE_ANON_KEY}`,

};

const isDev = (import.meta as any).env?.DEV === true;

const proxyUrl = isDev ? edgeFunctionUrl : `${window.location.origin}/api/supabase`;

// Usa agent VPN se disponível

const fetchOptions: RequestInit = {

    method: 'POST',

    headers: { 'Content-Type': 'application/json' },

    body: JSON.stringify({

```

```

    targetUrl: edgeFunctionUrl,

    targetMethod: 'POST',

    targetHeaders,

    targetBody: JSON.stringify(payload),

  })

};

// Adiciona agent VPN se conectado

if (proxyAgent) {

  (fetchOptions as any).agent = proxyAgent;

}

const response = isDev

  ? await fetch(edgeFunctionUrl, {

    method: 'POST',

    headers: targetHeaders,

    body: JSON.stringify(payload),

    ...(proxyAgent && { agent: proxyAgent })

  })

  : await fetch(proxyUrl, fetchOptions);

if (!response.ok) {

  const txt = await response.text();

  console.error("[PROXY FAIL]", response.status, txt);

```

```

    throw new Error(`Proxy (${response.status}): ${txt}`);
  }

  const data = await response.json();

  if (data.code && data.code !== 200) {

    throw new Error(`${exchange.name || 'Exchange':} ${data.msg || JSON.stringify(data)}`);

  }

  return data;
}

export async function fetchMarketInfo(exchange: Exchange) {

  if (!exchange.apiKey) return { pairs: [], quoteAssets: [] };

  try {

    // Usa VPN para requisição de market info

    return await callBinanceProxy('/fapi/v1/exchangeInfo', 'GET', {}, exchange);

  } catch (error) {

    console.error('[MARKET INFO]', error);

    return { pairs: [], quoteAssets: [] };

  }

}

export async function fetchRealAccountData(exchange: Exchange):
Promise<RealAccountData> {

  if (!exchange.apiKey) return { totalBalance: 0, unrealizedPnL: 0, assets: [], isSimulated: false };

  // Route to Hyperliquid service

```

```
if (exchange.id === 'hyperliquid') return fetchHLAccountData(exchange);

try {

  // Usa VPN para requisição de account data

  const data = await callBinanceProxy('/fapi/v2/account', 'GET', {}, exchange);

  // ... resto do código

} catch (error) {

  console.error('[ACCOUNT DATA]', error);

  return { totalBalance: 0, unrealizedPnL: 0, assets: [], isSimulated: false };

}

}
```

4. UI Components para VPN

4.1 VPN Status Panel

Arquivo: `src/components/VPNStatusPanel1.tsx`

```
import React, { useState, useEffect } from 'react';

import { getVPNManager } from '../services/vpnService';

import { Shield, Wifi, WifiOff, AlertCircle, Globe } from 'lucide-react';

export function VPNStatusPanel() {

  const [status, setStatus] = useState(getVPNManager().getStatus());

  const [metrics, setMetrics] = useState(getVPNManager().getMetrics());

  const [isToggling, setIsToggling] = useState(false);
```

```

useEffect(() => {

  const interval = setInterval(() => {

    setStatus(getVPNManager().getStatus());

    setMetrics(getVPNManager().getMetrics());

  }, 5000);

  return () => clearInterval(interval);

}, []);

const handleToggle = async () => {

  setIsToggling(true);

  await getVPNManager().toggle();

  setStatus(getVPNManager().getStatus());

  setIsToggling(false);

};

const statusColor = {

  CONNECTED: 'text-green-400',

  DISCONNECTED: 'text-gray-400',

  CONNECTING: 'text-yellow-400',

  ERROR: 'text-red-400'

}[status.status];

const statusIcon = {

  CONNECTED: <Wifi className="w-5 h-5 text-green-400" />,

  DISCONNECTED: <WifiOff className="w-5 h-5 text-gray-400" />,

```

```

CONNECTING: <Wifi className="w-5 h-5 text-yellow-400 animate-pulse" />,

ERROR: <AlertCircle className="w-5 h-5 text-red-400" />

}[status.status];

return (

  <div className="bg-slate-800 border border-slate-700 rounded-lg p-4">

    <div className="flex items-center justify-between mb-4">

      <div className="flex items-center gap-2">

        <Shield className="w-5 h-5 text-blue-400" />

        <h3 className="font-semibold">VPN Status</h3>

      </div>

      <button

        onClick={handleToggle}

        disabled={!isToggling}

        className="px-3 py-1 bg-blue-600 hover:bg-blue-700 rounded text-sm font-medium
disabled:opacity-50"

        >

        {status.status === 'CONNECTED' ? 'Desconectar' : 'Conectar'}

      </button>

    </div>

    <div className="space-y-2">

      <div className="flex items-center justify-between">

        <span className="text-sm text-slate-400">Status:</span>

```

```
<div className="flex items-center gap-2">

  {statusIcon}

  <span className={`text-sm font-medium ${statusColor}`}>

    {status.status}

  </span>

</div>

</div>

{status.connectedServer && (

  <>

    <div className="flex items-center justify-between">

      <span className="text-sm text-slate-400">Provedor:</span>

      <span className="text-sm font-medium capitalize">

        {status.connectedServer.provider}

      </span>

    </div>

    <div className="flex items-center justify-between">

      <span className="text-sm text-slate-400">País:</span>

      <div className="flex items-center gap-2">

        <Globe className="w-4 h-4" />

        <span className="text-sm font-medium">

          {status.connectedServer.country}

        </span>

      </div>

    </div>

  </>

)
```

```
        </span>

      </div>

    </div>

    <div className="flex items-center justify-between">

      <span className="text-sm text-slate-400">Latência:</span>

      <span className="text-sm font-medium">

        {status.connectedServer.latency}ms

      </span>

    </div>

  </>

)}

<div className="flex items-center justify-between pt-2 border-t border-slate-700">

  <span className="text-sm text-slate-400">Uptime:</span>

  <span className="text-sm font-medium">{metrics.uptime.toFixed(1)}%</span>

</div>

{status.lastError && (

  <div className="mt-2 p-2 bg-red-900/20 border border-red-700/50 rounded text-xs text-red-300">

    {status.lastError}

  </div>

)}

</div>
```

```
    </div>

  );

}
```

4.2 VPN Settings Panel

Arquivo: `src/components/VPNSettingsPanel.tsx`

```
import React, { useState } from 'react';

import { getVPNManager } from '../services/vpnService';

import { Settings } from 'lucide-react';

export function VPNSettingsPanel() {

  const vpnManager = getVPNManager();

  const [country, setCountry] = useState('us');

  const handleCountryChange = async (newCountry: string) => {

    setCountry(newCountry);

    await vpnManager.changeCountry(newCountry);

  };

  return (

    <div className="bg-slate-800 border border-slate-700 rounded-lg p-4">

      <div className="flex items-center gap-2 mb-4">

        <Settings className="w-5 h-5 text-blue-400" />

        <h3 className="font-semibold">Configurações VPN</h3>

      </div>

      <div className="space-y-3">
```

```
<div>
```

```
<label className="block text-sm text-slate-400 mb-2">
```

```
  País (Mullvad)
```

```
</label>
```

```
<select
```

```
  value={country}
```

```
  onChange={(e) => handleCountryChange(e.target.value)}
```

```
  className="w-full px-3 py-2 bg-slate-700 border border-slate-600 rounded text-sm"
```

```
>
```

```
  <option value="us">&img alt="Flag of the United States" data-bbox="338 451 361 466"/> Estados Unidos</option>
```

```
  <option value="de">&img alt="Flag of Germany" data-bbox="338 489 361 504"/> Alemanha</option>
```

```
  <option value="se">&img alt="Flag of Sweden" data-bbox="338 527 361 542"/> Suécia</option>
```

```
  <option value="nl">&img alt="Flag of the Netherlands" data-bbox="338 565 361 580"/> Holanda</option>
```

```
  <option value="gb">&img alt="Flag of the United Kingdom" data-bbox="338 603 361 618"/> Reino Unido</option>
```

```
  <option value="fr">&img alt="Flag of France" data-bbox="338 641 361 656"/> França</option>
```

```
  <option value="jp">&img alt="Flag of Japan" data-bbox="338 679 361 694"/> Japão</option>
```

```
  <option value="sg">&img alt="Flag of Singapore" data-bbox="338 717 361 732"/> Singapura</option>
```

```
</select>
```

```
</div>
```

```
<div className="flex items-center gap-2 p-2 bg-blue-900/20 border border-blue-700/50
rounded text-xs text-blue-300">
```

```
<span>&img alt="Information icon" data-bbox="228 886 251 901"/> </span>
```

```
    <span>VPN é usado apenas para requisições de trading (Binance, Hyperliquid)</span>

  </div>

</div>

</div>

);
}
```

5. Integração no App.tsx

5.1 Inicialização VPN

Arquivo: `src/App.tsx` (Modificado)

```
import { initializeVPN } from './services/vpnService';

import { VPNStatusPanel } from './components/VPNStatusPanel';

import { VPNSettingsPanel } from './components/VPNSettingsPanel';

export default function App() {

  // ... código existente ...

  useEffect(() => {

    // Inicializa VPN na startup

    const vpnConfig = {

      enabled: localStorage.getItem('vpn_enabled') === 'true',

      provider: (localStorage.getItem('vpn_provider') as any) || 'mullvad',

      primaryProvider: 'mullvad' as const,
```

```
    fallbackProvider: 'tor' as const,

    autoFailover: true,

    country: localStorage.getItem('vpn_country') || 'us',

    protocol: 'socks5' as const,

    timeout: 10000

  };

  initializeVPN(vpnConfig);

  // Salva preferências

  localStorage.setItem('vpn_enabled', vpnConfig.enabled.toString());

  localStorage.setItem('vpn_provider', vpnConfig.provider);

  localStorage.setItem('vpn_country', vpnConfig.country || 'us');

}, []);

return (

  // ... código existente ...

  <div className="flex gap-4">

    <VPNStatusPanel />

    <VPNSettingsPanel />

  </div>

);

}
```

6. Instalação de Dependências

6.1 Packages Necessários

Instalar dependências de proxy

```
npm install http-proxy-agent https-proxy-agent socks-proxy-agent
```

Ou com pnpm

```
pnpm add http-proxy-agent https-proxy-agent socks-proxy-agent
```

6.2 Tipos TypeScript

```
npm install --save-dev @types/http-proxy-agent @types/socks-proxy-agent
```

7. Configuração de Segurança

7.1 Proteção de API Keys

Arquivo: `src/utils/securityUtils.ts`

```
/**
```

```
 * Encripta API key antes de armazenar
```

```
 */
```

```
export function encryptAPIKey(apiKey: string): string {
```

```
    // Usa SubtleCrypto para encriptação
```

```
    const encoder = new TextEncoder();
```

```
    const data = encoder.encode(apiKey);
```

```
    // Implementar encriptação AES-256-GCM
```

```

    // Por enquanto, usa base64 como placeholder

    return btoa(apiKey);

}

/**
 * Decrypta API key
 */

export function decryptAPIKey(encrypted: string): string {

    return atob(encrypted);

}

/**
 * Mascara API key para exibição
 */

export function maskAPIKey(apiKey: string): string {

    if (apiKey.length <= 8) return '****';

    return apiKey.substring(0, 4) + '****' + apiKey.substring(apiKey.length - 4);

}

/**
 * Valida que VPN está conectado antes de usar API key
 */

export function validateVPNBeforeAPICall(): boolean {

    const vpnManager = getVPNManager();

    const status = vpnManager.getStatus();

```

```
if (status.status !== 'CONNECTED') {  
    throw new Error('VPN não está conectado. Conecte antes de usar API keys.');
```



```
}  
  
return true;  
}
```

8. Monitoramento e Logs

8.1 VPN Logging Service

Arquivo: `src/services/vpnLoggingService.ts`

```
interface VPNLog {  
    timestamp: Date;  
    event: string;  
    provider: string;  
    status: string;  
    latency?: number;  
    error?: string;  
}  
  
class VPNLogger {  
    private logs: VPNLog[] = [];
```

```
private maxLogs = 1000;

log(event: string, provider: string, status: string, latency?: number, error?: string) {

    const logEntry: VPNLog = {

        timestamp: new Date(),

        event,

        provider,

        status,

        latency,

        error

    };

    this.logs.push(logEntry);

    // Mantém apenas últimos 1000 logs

    if (this.logs.length > this.maxLogs) {

        this.logs = this.logs.slice(-this.maxLogs);

    }

    // Log no console em desenvolvimento

    if (import.meta.env.DEV) {

        console.log('[VPN LOG]', logEntry);

    }

}

getLogs(limit: number = 100): VPNLog[] {
```

```
        return this.logs.slice(-limit);
    }

    exportLogs(): string {
        return JSON.stringify(this.logs, null, 2);
    }

    clearLogs() {
        this.logs = [];
    }
}

export const vpnLogger = new VPNLogger();
```

9. Testes

9.1 Testes de Conectividade

Arquivo: `src/services/__tests__/vpnService.test.ts`

```
import { describe, it, expect, beforeAll } from 'vitest';
```

```
import VPNManager from '../vpnService';
```

```
describe('VPN Manager', () => {
```

```
    let vpnManager: VPNManager;
```

```
    beforeAll(() => {
```

```
        vpnManager = new VPNManager({
```

```
            enabled: true,
```

```
    provider: 'mullvad',

    primaryProvider: 'mullvad',

    fallbackProvider: 'tor',

    autoFailover: true,

    protocol: 'socks5',

    timeout: 10000

  });

});

it('deve conectar ao Mullvad', async () => {

  const result = await vpnManager.connect();

  expect(result).toBe(true);

  expect(vpnManager.getStatus().status).toBe('CONNECTED');

});

it('deve retornar status correto', () => {

  const status = vpnManager.getStatus();

  expect(status).toHaveProperty('provider');

  expect(status).toHaveProperty('status');

  expect(status).toHaveProperty('connectedServer');

});

it('deve testar latência', async () => {

  const status = vpnManager.getStatus();

  if (status.connectedServer) {
```

```
    expect(status.connectedServer.latency).toBeGreaterThan(0);

  }

});

it('deve desconectar', async () => {

  await vpnManager.disconnect();

  expect(vpnManager.getStatus().status).toBe('DISCONNECTED');

});

it('deve fazer fallback em caso de erro', async () => {

  // Simula erro no Mullvad

  vpnManager.getStatus().status = 'ERROR';

  const result = await vpnManager.connect();

  // Deve tentar fallback

  expect([true, false]).toContain(result);

});

});
```

10. Documentação de Uso

10.1 Guia do Usuário

Como ativar VPN:

1. Abra a aplicação

2. Clique no painel "VPN Status"
3. Clique em "Conectar"
4. Aguarde conexão (geralmente < 5 segundos)
5. Confirme que status mudou para "CONNECTED"

Como mudar país:

1. Abra "Configurações VPN"
2. Selecione novo país
3. VPN reconectará automaticamente

Resolução de Problemas:

Problema	Solução
Conexão lenta	Mude para país diferente ou use Tor
Erro ao conectar	Verifique internet, tente fallback
API key exposto	Reconecte VPN e regenere chaves

11. Cronograma de Implementação

Fase	Tarefa	Tempo	Status
1	Criar VPN Manager Service	4h	
2	Integrar com Exchange Service	2h	
3	Criar UI Components	3h	
4	Implementar Logging	1h	
5	Testes e Validação	3h	
6	Documentação	1h	
Total		14h	

12. Impacto na Segurança

Antes (Sem VPN)

Seu IP → ISP → Binance/Hyperliquid



Visível para todos

Depois (Com VPN)

Seu IP → Mullvad/Tor → Binance/Hyperliquid



IP Anônimo (Mullvad/Tor)

Benefícios:  IP oculto  Tráfego encriptado  Contorna bloqueios geográficos 
Proteção contra DDoS  Privacidade de dados sensíveis

13. Considerações de Performance

Impacto de Latência

Cenário	Latência Adicional
Sem VPN	0ms
Mullvad (mesmo país)	20-50ms
Mullvad (país distante)	50-150ms
Tor	200-500ms

Recomendação: Use Mullvad para trading (latência baixa), Tor apenas como fallback.

14. Conformidade Legal

 **Importante:** VPN é legal em maioria dos países, mas:

- Alguns países restringem VPN
- Algumas corretoras bloqueiam IPs de VPN
- Sempre verifique termos de serviço da corretora

Sugestão: Implemente detecção de bloqueio de VPN e avise usuário.

Conclusão

A integração de VPN proposta oferece:

✓ **Segurança:** Encriptação de tráfego ✓ **Privacidade:** IP oculto ✓ **Confiabilidade:** Fallback automático ✓ **Performance:** Latência mínima ✓ **Facilidade:** UI simples ✓ **Gratuito:** Mullvad sem custo

Próximo Passo: Implementar Fase 1 (VPN Manager Service)

Fim da Documentação VPN